

Implement Parallel Execution and Visual Checks in Playwright

Objective

This activity strengthens your test suite by adding speed and depth—through parallel execution and visual regression checks. You'll catch subtle UI issues that functional tests might miss, while improving runtime efficiency. Are your tests just passing—or truly protecting the user experience? This is your step toward faster, smarter, and more reliable automation.

Tools used: Playwright

Scenario

You've been asked to improve the speed and reliability of your Playwright test suite. Currently, it runs sequentially and checks only functional correctness—ignoring subtle layout regressions.

In this lab, you'll:

- * Enable parallel execution for faster test runs
- * Add visual checks using Playwright's built-in screenshot comparison
- * Simulate a UI change manually in your test application to trigger a visual diff

Your Mission

Improve your test suite by:

- * Running tests in parallel using multiple workers
- * Adding screenshot assertions to key UI areas
- * Validating that visual regressions are caught and reported

Instructions for learners

Task 1 – Enable Parallel Execution

1. In `playwright.config.ts`, configure parallelism:

```
``ts
```

```
workers: 4
```

```
``
```

2. Run the test suite before and after enabling parallel mode:

```
``bash
```

```
npx playwright test
```

```
``
```

3. Record and compare the execution time. Note any speed improvements.

Task 2 – Add Visual Screenshot Assertions

1. In at least two UI tests (e.g., login or dashboard), add screenshot assertions:

```
```ts
await expect(page).toHaveScreenshot('login-baseline.png');
```
```

2. Run the tests once to generate baseline screenshots:

```
```bash
npx playwright test
```
```

These images will serve as your visual reference going forward.

Task 3 – Simulate a Visual Change in the App

→ Instead of using multiple environments, make a ****small UI change in your local or test app**** to trigger a visual diff.

→ Examples:

→ * Change a CSS property (e.g., margin or font size)

→ * Move a button slightly or hide/show an element

* Inject a quick style override from within the test:

```
```ts
```

```
await page.addStyleTag({ content: 'button { margin-top: 50px }' });
```

```
```
```

3. Re-run your tests:

```
```bash
```

```
npx playwright test
```

```
```
```

Playwright should detect the layout change and fail the visual assertion.

Completion Criteria

- * Parallel execution is configured and running with `workers`:
- * Visual checks using `to Have Screenshot()` are added in key UI flows
- * A UI change was made to simulate a regression
- * Playwright detects the visual change and shows a diff
- * Learner reviews the diff in the HTML report

Reflection

Prompts

- * How much faster did your suite run with parallel execution?
- * How accurately did Playwright detect visual changes?
- * How easy was it to interpret the failure report?
- * Would functional assertions alone have missed this issue?
- * When might you avoid visual testing?
- (e.g., animations, dynamic graphs, real-time counters)

Submission

Requirements

- * Snippet of your updated `playwright.config.ts`
- * Screenshot or summary of the visual diff report
- * A short write-up (~150 words) covering:
 - * What you changed visually
 - * How Playwright responded
 - * Whether this improved confidence in your test coverage

Conclusion

Next Step

→ You've now integrated speed and UI validation into your suite. Visual tests help catch issues functional assertions miss—and parallelism ensures it happens fast.

→ In the next step, Coursera Coach will help you reflect on how to scale this across your full test strategy and CI pipelines.